DELTA EXCHANGE TRADING BOT

Complete Project Plan, Strategy Roadmap & To-Do Checklist

Built with Claude AI • Non-Coder Edition • Local to Production

~97 Days Total timeline	**13 Strategies 5 existing + 8 new**	**$20/month Claude Pro only**	**$0 Cloud Until paper trading done**

1. Project Overview

This document is the complete working plan to take the Delta Exchange Trading Bot from its current state to a production-grade, professionally deployed algorithmic trading system. It is designed for a non-coder working with Claude AI as their technical partner.

1.1 What we are building

- An algorithmic trading bot for Delta Exchange India (crypto perpetual futures)
- Supports paper trading (no real money) and live trading (real money)
- 13 total strategies covering all 4 market regimes: trending, ranging, high-volatility, low-volatility
- Full observability: Prometheus metrics + Grafana dashboards
- Runs on your local Windows machine, later migrates to cheap cloud (Hetzner ~₹3.79/month)

1.2 Your tools and costs

Tool	Cost	Purpose
Claude Pro	$20 / month	Your AI coding partner. Write code, debug, explain everything.
VS Code	Free	Code editor. Where you view and edit files.
Python 3.12	Free	The language the bot is written in.
Git	Free	Version control. Saves history of all changes.
Docker Desktop	Free	Runs databases and monitoring locally.
Hetzner Cloud	~₹320/month	Cloud server when ready for live trading. NOT needed now.

> Important: Do NOT spend money on AWS, Azure, or any other cloud service until the bot has run in paper mode successfully for at least 2 weeks. Everything in Phases 1-3 runs on your local machine at zero cloud cost.

2. Complete Project Timeline

Estimates are based on 2-3 hours of active work per day with Claude AI. If you work more, you finish faster. If life gets busy, the estimates stretch. There is no rush - getting it right matters more than getting it done fast.

Phase	Name	Days	Goal
Phase 1	Local setup + running	Days 1-7	Bot running in paper mode on your machine
Phase 2	Fix critical issues	Days 8-37	Security fixed, bugs corrected, stops on exchange
Phase 3	8 new strategies	Days 38-82	Full 13-strategy regime-aware bot
Phase 4	Paper validation	Days 83-97	2 weeks paper trading before any real money
Phase 5	Cloud migration (optional)	Days 98+	Move to Hetzner, run 24/7, go live

Phase 1: Local Setup & First Run (Days 1-7)

> Goal: See the bot running and printing live paper trades within your first week.

Tasks:

- Check Python version: run python --version in PowerShell. Must be 3.12.x
- Install Poetry (project package manager) - one command, takes 5 minutes
- Clone the project from GitHub onto your Desktop
- Run: poetry install - downloads all libraries automatically
- Create a safe .env file replacing the leaked API keys
- Run the bot: poetry run python scripts/run_bot.py --mode paper --strategy portfolio
- Observe log output for 48-72 hours. Note any errors or odd behaviour
- Register on Delta Exchange testnet, get clean API keys, update .env

Done when:

- Bot runs without crashing
- Log lines appear every 60 seconds showing signal generation
- state.db file exists on your machine with trade records inside

Phase 2: Fix Critical Issues (Days 8-37)

> Goal: Fix the 8 known bugs and security issues before adding any new features.

Task	Days	Why it matters
Fix Dockerfile Python 3.11 -> 3.12	0.5 days	Docker build breaks without this fix
Merge duplicate strategy packages	3 days	Two RSIScalping classes cause confusion and bugs
Wire YAML config files to actually load	2 days	config/paper.yml is currently ignored completely
Fix RSI Wilder's smoothing formula	1 day	Current RSI is ~7 points off vs all charting tools
Place real stop-loss orders on exchange	6 days	Memory-only stops are lost if bot crashes
Add funding rate awareness	4 days	Perpetual funding costs are silently ignored now
Add max holding period per position	2 days	Positions can stay open forever if stop never hits
Add Grafana visual dashboard	10 days	See PnL, drawdown, win rate on a screen, not log files

Phase 3: New Advanced Strategies (Days 38-82)

> Your valid point: the existing 5 strategies ignore volume, multi-timeframe direction, order book pressure, and crypto-specific signals like funding rates. These 8 additions directly fix those gaps.

Strategy	Days	Regime	What it adds
VWAP Deviation	5 days	RANGING / LOW_VOL	Volume-weighted price level - most-watched by pros
Bollinger Bands Squeeze	4 days	LOW_VOL transition	Detects volatility coiling before explosive breakouts
MACD Histogram Divergence	4 days	TRENDING	Momentum + trend combined, earlier entries than EMA
Order Book Imbalance	6 days	All regimes	Signals BEFORE price moves using L2 order data
Multi-Timeframe Confluence	6 days	All (filter)	1m + 15m + 1h must agree - halves false signals
Funding Rate Extremes	6 days	HIGH_VOL	Crypto-unique contrarian signal from perpetual funding
Volume-Weighted Momentum	4 days	All regimes	Replaces weak 3-bar momentum with volume filter
Support / Resistance Breakout	5 days	TRENDING / HIGH_VOL	20-bar high/low breakout with volume confirmation
Backtest all new strategies	5 days	N/A	Validate each strategy before adding to live dispatch

Phase 4: Paper Trading Validation (Days 83-97)

> MANDATORY: Never use real money until paper trading meets these thresholds for 2 full weeks: Sharpe ratio above 0.8, max drawdown below 8%, win rate above 45%.

- Run fully upgraded bot in paper mode for 14 consecutive days
- Monitor Grafana dashboard daily - check PnL, drawdown, signal quality
- Run scripts/analyze_paper_trades.py weekly for per-strategy breakdown
- Tune parameters on underperforming strategies using Claude's guidance
- Only proceed to live trading after 2 solid weeks of good paper performance

3. Existing Strategies Analysis

Strategy	Rating	Status	Regime	Action needed
Trend Following	4/5	Keep	TRENDING	Minor: add volume confirmation later
Mean Reversion	4/5	Keep	RANGING	Minor: upgrade to Bollinger variant later
EMA Crossover	3/5	Keep + minor fix	TRENDING	Add volume filter in Phase 3
RSI Scalping	3/5	Fix RSI formula	RANGING / HIGH_VOL	Fix Wilder's smoothing in Phase 2
Momentum (3-bar)	2/5	Replace	All regimes	Replace with Volume-Weighted Momentum in Phase 3

4. Complete To-Do Checklist

Use this checklist to track progress. Check off each item as you complete it with Claude's help. Never skip items - each one builds on the previous.

PHASE 1: Local Setup (Days 1-7)

Day 1 - Installation

- [ ] Open PowerShell (Win + X, click Terminal)
- [ ] Run: python --version -- must show 3.12.x
- [ ] If not 3.12: tell Claude and upgrade Python first
- [ ] Run: poetry --version -- must show 1.8.x or higher
- [ ] If not installed: run the Poetry install command from Claude

Day 1-2 - Get the project

- [ ] Run: cd $env:USERPROFILE\Desktop
- [ ] Run: git clone https://github.com/vinugani/Trading_Algo.git
- [ ] Run: cd Trading_Algo
- [ ] Run: code . -- VS Code opens with project files
- [ ] Run: poetry install -- wait for all libraries to download
- [ ] Confirm no red errors in poetry install output

Day 2 - Security first

- [ ] Open .env file in VS Code
- [ ] DELETE all existing content (old leaked keys)
- [ ] Paste the safe paper-mode .env template from Claude
- [ ] Save with Ctrl+S
- [ ] Register at: cdn-ind.testnet.deltaex.org
- [ ] Generate testnet API key and secret
- [ ] Update .env with your testnet key and secret

Day 2 - First run

- [ ] Run: poetry run python scripts/run_bot.py --mode paper --strategy portfolio
- [ ] See JSON log lines appear -- bot is running!
- [ ] Let it run for at least 30 minutes
- [ ] Press Ctrl+C to stop
- [ ] Open state.db in VS Code with SQLite Viewer extension
- [ ] Confirm there are rows in the trades or signals table

Days 3-7 - Observe

- [ ] Run bot each day and let it run for a few hours in paper mode
- [ ] Note any error messages (paste to Claude to fix)
- [ ] Understand what the log output means (ask Claude to explain any line)
- [ ] Install SQLite Viewer extension in VS Code
- [ ] PHASE 1 COMPLETE - tick this when bot runs without errors for 2+ days

PHASE 2: Fix Critical Issues (Days 8-37)

Security fixes (Day 8)

- [ ] Open Dockerfile, change python:3.11-slim to python:3.12-slim, save
- [ ] Verify: tell Claude you made the change and run the validation command

Code cleanup (Days 8-15)

- [ ] Merge strategy/ and strategies/ packages (Claude writes all code)
- [ ] Update all imports that reference the old strategy/ package
- [ ] Run: poetry run pytest tests/ -- all tests should still pass
- [ ] Wire config/paper.yml to be auto-loaded by Settings class
- [ ] Wire config/prod.yml for live mode
- [ ] Test that changing values in YAML actually changes bot behaviour

Bug fixes (Days 12-15)

- [ ] Fix RSI formula: replace simple average with Wilder's smoothing
- [ ] Verify RSI values match TradingView for the same candle data
- [ ] Test RSI fix with: poetry run pytest tests/unit/test_rsi_scalping_strategy.py

Exchange stop orders (Days 14-20)

- [ ] Add place_stop_limit_order() method to DeltaClient
- [ ] Store exchange stop order ID in open_position_state table
- [ ] Place stop order on exchange after every entry
- [ ] Cancel exchange stop order when bot closes a position
- [ ] Test in paper mode for 24 hours -- confirm stops appear in testnet account

Funding rate (Days 20-24)

- [ ] Add get_funding_rate() method to DeltaClient
- [ ] Log current funding rate to execution_logs metadata on each trade
- [ ] Add funding_rate_cost to the fee calculation in FeeManager
- [ ] Confirm PnL calculation now includes funding costs

Max holding period (Days 24-26)

- [ ] Add max_hold_candles setting to Settings (default: 50 candles = 50 minutes on 1m)
- [ ] Track entry time in open_position_state
- [ ] Auto-close position if open longer than max_hold_candles
- [ ] Test: create a paper position and verify it auto-closes after time limit

Grafana dashboard (Days 26-37)

- [ ] Add Grafana service to docker-compose.yml
- [ ] Add Loki log aggregation to docker-compose.yml
- [ ] Run: docker compose up -d
- [ ] Open http://localhost:3000 -- Grafana login screen
- [ ] Connect Prometheus data source (url: http://prometheus:9090)
- [ ] Import Trading Overview dashboard (JSON provided by Claude)
- [ ] Import Risk Monitor dashboard
- [ ] Import API Health dashboard
- [ ] Confirm live bot data flows into dashboards
- [ ] PHASE 2 COMPLETE - bot is secure, bugs fixed, dashboards working

PHASE 3: New Strategies (Days 38-82)

Strategy 1: VWAP Deviation (Days 38-42)

- [ ] Claude writes VWAPDeviationStrategy class in strategies/vwap_deviation.py
- [ ] Add VWAP calculation using volume column from candles DataFrame
- [ ] Entry signal: price > 0.5% below VWAP = buy; > 0.5% above = sell
- [ ] Add to StrategyManager dispatch for RANGING regime
- [ ] Backtest against 30 days of BTCUSD 1m data
- [ ] Confirm Sharpe ratio > 0.5 before activating
- [ ] Run in paper mode for 48 hours

Strategy 2: Bollinger Bands Squeeze (Days 43-46)

- [ ] Claude writes BollingerSqueezeStrategy class
- [ ] Calculate BB width = (upper - lower) / middle band
- [ ] Detect squeeze: BB width < 20-bar average of BB width
- [ ] Entry: breakout above upper band with volume spike = buy
- [ ] Add to StrategyManager for LOW_VOL to HIGH_VOL transition
- [ ] Backtest and validate

Strategy 3: MACD Histogram (Days 47-50)

- [ ] Claude writes MACDHistogramStrategy class
- [ ] Calculate MACD(12,26,9) using EMA functions already in codebase
- [ ] Entry: histogram turning positive + signal line cross = buy
- [ ] Add to StrategyManager for TRENDING regime alongside trend_following
- [ ] Backtest and validate

Strategy 4: Order Book Imbalance (Days 51-56)

- [ ] Claude writes OrderBookImbalanceStrategy class
- [ ] Use existing get_orderbook() from DeltaClient
- [ ] Calculate: bid_volume / (bid_volume + ask_volume) in top 5 levels
- [ ] Entry: imbalance > 0.70 = buy signal; < 0.30 = sell signal
- [ ] Add to StrategyManager for all regimes as secondary signal
- [ ] Backtest and validate

Strategy 5: Multi-Timeframe Confluence (Days 57-62)

- [ ] Claude writes MultiTimeframeFilter class
- [ ] Fetch 15m candles using existing fetch_candles(symbol, '15m')
- [ ] Fetch 1h candles using fetch_candles(symbol, '1h') -- needs adding to market_data.py
- [ ] Only pass signal if 1m, 15m, and 1h all agree on direction
- [ ] Apply as a filter wrapper around all existing strategies
- [ ] Test: verify fewer but higher-quality signals in paper mode

Strategy 6: Funding Rate Extremes (Days 63-68)

- [ ] Claude writes FundingRateStrategy class
- [ ] Use get_funding_rate() added in Phase 2
- [ ] Entry: funding > +0.1% (heavily long crowded) = sell signal
- [ ] Entry: funding < -0.05% (heavily short crowded) = buy signal
- [ ] Add to StrategyManager for HIGH_VOL regime
- [ ] Backtest and validate

Strategy 7: Volume-Weighted Momentum (Days 69-72)

- [ ] Claude writes VolumeWeightedMomentumStrategy class
- [ ] Replace old 3-bar momentum with 10-bar momentum
- [ ] Multiply signal confidence by volume_ratio = current_volume / avg_volume_20
- [ ] Delete old MomentumStrategy from legacy strategy/ package
- [ ] Backtest: compare vs old momentum -- must be better

Strategy 8: S/R Breakout (Days 73-77)

- [ ] Claude writes SupportResistanceBreakoutStrategy class
- [ ] Calculate 20-bar high (resistance) and 20-bar low (support)
- [ ] Entry: close > 20-bar high AND volume > 1.5x average = buy
- [ ] Entry: close < 20-bar low AND volume > 1.5x average = sell
- [ ] Add to StrategyManager for TRENDING and HIGH_VOL
- [ ] Backtest and validate

Full backtest validation (Days 78-82)

- [ ] Run BacktestEngine on each new strategy individually
- [ ] Only add strategy to live regime dispatch if Sharpe > 0.8
- [ ] Run all 13 strategies together in paper mode for 72 hours
- [ ] Check Grafana -- confirm per-strategy PnL shows positive expectancy
- [ ] PHASE 3 COMPLETE - 13 strategies, all validated

PHASE 4: Paper Trading Validation (Days 83-97)

Mandatory validation gates

- [ ] Run full 13-strategy bot in paper mode for 14 consecutive days
- [ ] Day 7 checkpoint: Sharpe ratio > 0.5, drawdown < 6%
- [ ] Day 14 checkpoint: Sharpe ratio > 0.8, drawdown < 8%, win rate > 45%
- [ ] Run: poetry run python scripts/analyze_paper_trades.py
- [ ] Review per-strategy breakdown -- identify any losing strategy
- [ ] If a strategy consistently loses: disable it (ask Claude to help tune or disable)
- [ ] PHASE 4 COMPLETE - all validation gates passed

PHASE 5: Cloud Migration (Days 98+)

Hetzner server setup

- [ ] Register at hetzner.com (no credit card required for signup)
- [ ] Create CX22 server: 2 vCPU, 4GB RAM, Ubuntu 22.04, ~3.79 EUR/month
- [ ] Set up SSH key for secure access (Claude guides you through this)
- [ ] Connect to server with SSH from Windows PowerShell
- [ ] Install Python 3.12, Git, Docker on the server (same steps as local)
- [ ] Copy project files to server using git clone
- [ ] Set up .env with production API keys (NOT testnet)
- [ ] Run final pre-flight check: poetry run python scripts/live_preflight.py
- [ ] Start bot on server in paper mode first, confirm it works

Going live (do not rush this)

- [ ] Paper mode passes on cloud server for 48 hours
- [ ] Set DELTA_MODE=live in .env on server
- [ ] Set DELTA_ORDER_SIZE=50 (start small - half the paper size)
- [ ] Start live bot with: nohup poetry run python scripts/run_bot.py --mode live &
- [ ] Monitor Grafana for the first 4 hours intensively
- [ ] Confirm first real trade executes correctly on Delta Exchange
- [ ] Set up systemd service for auto-restart on server reboot
- [ ] Set up Alertmanager alerts to your phone/Telegram
- [ ] PHASE 5 COMPLETE - production live trading system

5. How to Work With Claude Effectively

These rules will make your sessions faster and more productive. The better you describe what you want, the better code you get.

5.1 The golden workflow

Step	Action	Example
1	Describe in plain language	"I want the bot to stop trading if the market is very quiet - less than 0.1% movement in the last hour"
2	Claude writes the code	Claude produces a complete Python file with explanation
3	You copy-paste the code	Open VS Code, find the file, paste the code exactly as given
4	Run and test	Run the bot or the test command Claude gives you
5	Paste errors back	If you see red text, copy the whole error message and paste it to Claude
6	Iterate until it works	Keep going until the output looks right. No limit on attempts.

5.2 Tips for better results

- Always paste the FULL error message, not just the last line
- Tell Claude which file you are editing and what line you are on
- If a response is confusing, say: 'explain this like I am not a programmer'
- Save your work before every change: Ctrl+S in VS Code
- Use Claude Projects and upload your key files for persistent context
- When something works, write a short note in a text file so you remember what you did

5.3 Context window management

> In long sessions (2+ hours), Claude may start to lose context from the start of the conversation. Signs: Claude asks you to re-explain something it knew earlier, or gives code that contradicts something it said before. Fix: start a fresh conversation, paste the relevant file contents, and continue.

- Use Claude Projects to store persistent context across sessions
- Upload your most-edited files to the Project at the start of each week
- Break large tasks into smaller daily sessions rather than one very long session

6. Known Issues to Fix (Reference)

These are confirmed bugs discovered during code analysis. Each is addressed in the checklist above. This section is for reference.

Sev.	Issue	Location	Fix in
CRIT	Real API keys committed to GitHub	.env file	Phase 1 Day 2
CRIT	Stops/TPs only in memory - lost on crash	order_execution_engine.py	Phase 2
HIGH	Dockerfile uses Python 3.11, needs 3.12	Dockerfile line 1	Phase 2 Day 8
HIGH	RSI uses wrong formula (7 points off)	strategy/rsi_scalping.py	Phase 2 Day 12
HIGH	Duplicate strategy/ and strategies/ packages	src/ directory	Phase 2 Day 8
MED	YAML config files not loaded by Settings	core/settings.py	Phase 2 Day 10
MED	Funding rate costs completely ignored	execution/fee_manager.py	Phase 2 Day 20
MED	No max holding period for positions	core/engine.py	Phase 2 Day 24
MED	Redis and PostgreSQL declared but not wired	persistence/db.py	Phase 5 (cloud)
LOW	No Grafana dashboards	monitoring/	Phase 2 Day 26
LOW	No CI/CD pipeline	No .github/workflows/	Phase 5 (cloud)

7. Final Notes & Reminders

Never trade real money until paper mode has run successfully for 2 weeks and all Phase 4 validation gates have passed. Losing real money to a preventable bug is the worst outcome. There is no rush.

Always start with the smallest possible position size when going live. Start at half your paper size ($50 if paper was $100). You can increase size once you see the live bot behaving correctly.

This is a working system. Strategies will lose sometimes - that is normal. What matters is overall statistical edge over many trades. A single losing trade does not mean the bot is broken.

Monthly cost summary

Item	Cost / Month	Notes
Claude Pro	$20	Your AI coding partner. Essential.
Local development (Phases 1-4)	$0	All runs on your own machine.
Hetzner CX22 (Phase 5 only)	~₹320 (~$4)	Only needed for 24/7 live trading.
Delta Exchange trading fees	0.02-0.05% per trade	Maker 0.02%, taker 0.05%.
TOTAL during development	$20	Just Claude Pro.
TOTAL when live on cloud	~$24	Claude Pro + Hetzner.

Document prepared with Claude AI • Trading Bot Project Plan • Version 1.0